

LAPORAN TUGAS 7

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

Tanggal : 17-12-2025

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DAFTAR ISI

DAFTAR ISI.....	i
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Tujuan Praktikum.....	1
1.3. Spesifikasi Sistem	1
BAB II DASAR TEORI	2
2.1. Memory Management.....	2
2.2. Address Space	2
BAB III JAWABAN TUGAS PRAKTIKUM.....	4
3.1.1. Tugas 1	4
3.1.2. Tugas 2.....	4
3.1.3. Tugas 3.....	4
3.1.4. Tugas 4.....	4
3.1.5. Tugas 5.....	4
3.1.6 Tugas 6.....	4
3.2. Source Code	5
3.2.1. Source Tugas 1.....	5
3.2.2. Source Tugas 2.....	7
3.2.3. Source Tugas 3.....	12
3.2.4. Source Tugas 4.....	16
3.2.5. Source Tugas 5.....	23
3.3. Screenshot Output.....	28
3.3.2. Tugas 2.....	29
3.3.3. Tugas 3.....	30
3.3.4. Tugas 4.....	30
3.3.5. Tugas 5.....	31
3.4. Penjelasan Implementasi.....	34
3.4.1. Tugas 1.....	34
3.4.2. Tugas 2.....	34
3.4.3. Tugas 3.....	35
3.4.3. Tugas 4	35

3.4.5. Tugas 5.....	36
3.4.6. Tugas 6.....	36
BAB IV ANALISIS DAN PEMBAHASAN.....	37
4.1. Analisis Tugas 1.....	37
4.2. Analisis Tugas 2.....	37
4.3. Analisis Tugas 3.....	38
4.4. Analisis Tugas 4.....	38
4.5. Analisis Tugas 5.....	39
4.6. Analisis Tugas 6.....	39
BAB V KESIMPULAN DAN SARAN	40
5.1. Kesimpulan	40
5.2. Saran	41
5.3. Kendala yang dihadapi.....	41
DAFTAR PUSTAKA	43

BAB I PENDAHULUAN

1.1. Latar Belakang

Manajemen memory merupakan salah satu komponen terpenting dalam sistem operasi karena berhubungan langsung dengan pengelolaan sumber daya memori utama (main memory). Memory utama digunakan untuk menyimpan data dan intruksi yang sedang dieksekusi oleh prosesor. Pengelolaan memori yang tidak efisien dapat menyebabkan penurunan kinerja sistem, pemborosan sumber daya, bahkan kegagalan proses.

Seiring berkembangnya sistem komputer modern, teknik manajemen memori juga mengalami perkembangan, seperti penggunaan paging, swapping, serta multi-level page table. Selain itu, perbedaan arsitektur, proses seperti intel dan ARM turut memengaruhi cara sistem operasi mengelola memori.

Oleh karena itu, melalui praktikum main memory ini, mahasiswa diharapkan dapat memahami konsep dasar manajemen memori secara teoritis maupun praktis, serta mampu menganalisis cara kerja pengelolaan memori pada sistem operasi modern.

1.2. Tujuan Praktikum

1. Memahami konsep dasar manajemen memori pada sistem operasi.
2. Mengetahui pembagian address space pada proses (text, data, brs, heap, dan stack).
3. Memeahami mekanisme alokasi memori secara kontigu menggunakan algoritma tertentu.
4. Mengetahui cara kerja paging dan two-level page table.
5. Memahami konsep swapping serta strategi yang digunakan

1.3. Spesifikasi Sistem

1. Perangkat Keras (Hardware)
 - Prosesor : AMD Ryzen (64-bit)
 - Ram : 4 GB
2. Perangkat Lunak (Software)
 - Sistem Operasi : Linux (Ubuntu 20.04 / 22.04 atau setara)
 - Compiler : GCC (GNU Compiler Collection)
 - Editor : Visual Studio Code
 - Shell : Bash

BAB II DASAR TEORI

2.1. Memory Management

Memory management adalah salah satu fungsi utama sistem operasi yang bertugas mengatur penggunaan memori utama (main memory). Sistem operasi harus memastikan bahwa setiap proses mendapatkan ruang memori yang cukup, tidak saling mengganggu, serta memanfaatkan memori secara efisien.

Memory management meliputi proses alokasi, dealokasi, pemetaan alamat logis ke alamat fisik, serta perlindungan memori antar proses.

2.2. Address Space

Address space adalah sekumpulan alamat memori yang dapat diakses oleh suatu proses. Setiap proses memiliki address space sendiri sehingga tidak dapat mengakses memori proses lain secara langsung.

Address space umumnya terbagi menjadi beberapa bagian, yaitu:

- Text Segment (kode program)
- Data Segment (variabel global terinisialisasi)
- BSS Segment (variabel global terinisialisasi)
- Heap (alokasi dinamis)
- Stack (variabel lokal dan pemanggilan fungsi)

2.3. Contiguous Memory Allocation

Contiguous memory allocation adalah metode pengelolaan memori di mana setiap proses dialokasikan pada satu blok memori yang bersebelahan (kontigu). Metode ini sederhana dan mudah diimplementasikan, namun dapat menimbulkan fragmentasi eksternal.

Algoritma yang umum digunakan dalam contiguous allocation antara lain:

1. First-Fit
2. Best-Fit
3. Worst-Fit

2.4. Paging

Paging adalah teknik memory management yang membagi memori fisik menjadi frame dan memori logis menjadi page dengan ukuran yang sama. Paging digunakan untuk mengatasi fragmentasi eksternal yang terjadi pada contiguous allocation.

Sistem operasi menggunakan page table untuk menerjemahkan alamat logis menjadi alamat fisik.

2.5. Two-Level-Paging Table

Pada sistem dengan address space besar, single-level page table membutuhkan memori yang besar. Oleh karena itu digunakan two-level page table untuk menghemat penggunaan memori.

Two-level paging membagi page table menjadi:

- Outer page table
- Inner page table

Page table hanya dibuat saat dibutuhkan.

2.6. Swapping

Swapping adalah teknik memory management di mana proses dipindahkan sementara dari main memory ke secondary storage (disk) ketika memori utama penuh. Proses akan dimuat kembali ke memori saat diperlukan.

Strategi swapping yang umum digunakan adalah FIFO, LRU, dan Priority-based swapping.

2.7. Arsitektur Memory Management (Intel & ARM)

Arsitektur prosesor memengaruhi implementasi memory management.

- Intel x86-64 menggunakan paging dengan 4 level page table
- ARMv8 mendukung beberapa ukuran page dan jumlah level paging yang fleksibel

BAB III JAWABAN TUGAS PRAKTIKUM

3.1. Deskripsi Tugas

3.1.1. Tugas 1

Tugas ini bertujuan untuk menganalisis layout memori proses pada sistem operasi Linux, meliputi text segment, data segment, BSS, heap, dan stack. Program dimodifikasi untuk menampilkan alamat berbagai jenis variabel, fungsi, alokasi memori dinamis, serta perilaku stack pada pemanggilan rekursif.

3.1.2. Tugas 2

Tugas ini bertujuan untuk membandingkan empat algoritma contiguous memory allocation, yaitu First-Fit, Best-Fit, Worst-Fit, dan Next-Fit berdasarkan waktu eksekusi dan fragmentasi eksternal.

3.1.3. Tugas 3

Tugas ini bertujuan untuk mengimplementasikan dan membandingkan algoritma page replacement FIFO dan LRU serta menganalisis performanya berdasarkan page fault, hit ratio, dan disk write.

3.1.4. Tugas 4

Tugas ini bertujuan membandingkan berbagai struktur page table, yaitu single-level, two-level, three-level, dan inverted page table berdasarkan memory overhead dan kecepatan translasi alamat.

3.1.5. Tugas 5

Tugas ini bertujuan untuk mensimulasikan mekanisme swapping pada sistem operasi dengan membandingkan tiga strategi pemilihan victim: FIFO, LRU, dan Priority-based.

3.1.6 Tugas 6

Tugas ini bertujuan untuk menganalisis arsitektur memori pada sistem nyata (Linux/Ubuntu), meliputi arsitektur CPU, page size, TLB, cache hierarchy, virtual memory, dan swapping, serta membandingkannya dengan arsitektur Intel IA-32, x86-64, dan ARMv8.

3.2. Source Code

3.2.1. Source Tugas 1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;    // Data segment
int uninit_var;          // BSS segment

// ----- Multiple Functions -----
void func1() {}
void func2() {}
void func3() {}

// ----- Recursive Function -----
void recursive_func(int level) {
    int local_var = level;
    printf("  Level %d local_var address: %p\n",
           level, (void*)&local_var);

    if (level > 0)
        recursive_func(level - 1);
}

int main() {
    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");
    printf("PID: %d\n\n", getpid());

    // ----- Text Segment -----
    printf("Text Segment:\n");
    printf("  main() address : %p\n", (void*)main);
```



```

printf(" func1() address: %p\n", (void*)func1);
printf(" func2() address: %p\n", (void*)func2);
printf(" func3() address: %p\n\n", (void*)func3);

// ----- Data Segment -----
static int static_array[5] = {1,2,3,4,5};
char *str_literal = "Hello Memory";

printf("Data Segment:\n");
printf(" global_var address : %p\n", (void*)&global_var);
printf(" static_array[0] address: %p\n", (void*)&static_array[0]);
printf(" string literal address : %p\n\n", (void*)str_literal);

// ----- Heap -----
int *dynamic_array = (int*)malloc(5 * sizeof(int));
int *block1 = (int*)malloc(100);
int *block2 = (int*)malloc(200);

printf("Heap Segment:\n");
printf(" dynamic_array[0] address: %p\n", (void*)&dynamic_array[0]);
printf(" malloc block 1 address : %p\n", (void*)block1);
printf(" malloc block 2 address : %p\n\n", (void*)block2);

// ----- Stack -----
int stack_var = 50;
printf("Stack Segment:\n");
printf(" stack_var address: %p\n\n", (void*)&stack_var);

// ----- Recursive Stack -----
printf("Stack (Recursive Calls):\n");
recursive_func(3);

```

```

printf("\n");

// ----- Memory Map -----
printf("Tekan Enter untuk melihat /proc/[pid]/maps...");
getchar();

char cmd[100];
sprintf(cmd, "cat /proc/%d/maps", getpid());
system(cmd);

// Cleanup
free(dynamic_array);
free(block1);
free(block2);

return 0;
}

```

3.2.2. Source Tugas 2

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

#define MAX_BLOCKS 20
#define MEMORY_SIZE 1000

typedef struct {
    int id;
    int size;
    int start;
    int allocated;

```

```
} Block;
```

```
Block mem[MAX_BLOCKS];
```

```
int block_count;
```

```
int last_alloc = 0; // untuk Next-Fit
```

```
void init_memory() {
```

```
    mem[0] = (Block){-1, MEMORY_SIZE, 0, 0};
```

```
    block_count = 1;
```

```
    last_alloc = 0;
```

```
}
```

```
void split_block(int idx, int pid, int size) {
```

```
    int old_size = mem[idx].size;
```

```
    int old_start = mem[idx].start;
```

```
    mem[idx] = (Block){pid, size, old_start, 1};
```

```
    if (old_size > size) {
```

```
        for (int j = block_count; j > idx + 1; j--)
```

```
            mem[j] = mem[j - 1];
```

```
        mem[idx + 1] = (Block){-1, old_size - size, old_start + size, 0};
```

```
        block_count++;
```

```
    }
```

```
}
```

```
int first_fit(int pid, int size) {
```

```

for (int i = 0; i < block_count; i++)
    if (!mem[i].allocated && mem[i].size >= size) {
        split_block(i, pid, size);
        return 1;
    }
return 0;
}

int best_fit(int pid, int size) {
    int best = -1, min = MEMORY_SIZE + 1;
    for (int i = 0; i < block_count; i++)
        if (!mem[i].allocated && mem[i].size >= size && mem[i].size < min) {
            min = mem[i].size;
            best = i;
        }
    if (best == -1) return 0;
    split_block(best, pid, size);
    return 1;
}

int worst_fit(int pid, int size) {
    int worst = -1, max = -1;
    for (int i = 0; i < block_count; i++)
        if (!mem[i].allocated && mem[i].size >= size && mem[i].size > max) {
            max = mem[i].size;
            worst = i;
        }
    if (worst == -1) return 0;

```

```

    split_block(worst, pid, size);
    return 1;
}

int next_fit(int pid, int size) {
    int count = 0;
    int i = last_alloc;

    while (count < block_count) {
        if (!mem[i].allocated && mem[i].size >= size) {
            split_block(i, pid, size);
            last_alloc = i;
            return 1;
        }
        i = (i + 1) % block_count;
        count++;
    }
    return 0;
}

void calculate_fragmentation() {
    int total_free = 0, holes = 0, largest = 0;
    for (int i = 0; i < block_count; i++) {
        if (!mem[i].allocated) {
            total_free += mem[i].size;
            holes++;
            if (mem[i].size > largest)
                largest = mem[i].size;
        }
    }
}

```

```

    }
}
float external_frag = (holes > 0)
    ? (float)(total_free - largest) / total_free * 100
    : 0;

printf("Total Free Space    : %d\n", total_free);
printf("Largest Free Block  : %d\n", largest);
printf("Free Holes          : %d\n", holes);
printf("External Fragmentation: %.2f%%\n", external_frag);
}

void benchmark(const char *name, int (*alloc)(int,int)) {
    struct timeval start, end;
    init_memory();

    gettimeofday(&start, NULL);

    alloc(1,100); alloc(2,200); alloc(3,300); alloc(4,250); alloc(5,150);
    alloc(6,180); alloc(7,220); alloc(8,100);

    gettimeofday(&end, NULL);

    long time = (end.tv_sec - start.tv_sec) * 1000000 +
        (end.tv_usec - start.tv_usec);

    printf("\n[%s]\nExecution Time: %ld μs\n", name, time);
    calculate_fragmentation();
}

```

```
}
```

```
int main() {  
    benchmark("First-Fit", first_fit);  
    benchmark("Best-Fit", best_fit);  
    benchmark("Worst-Fit", worst_fit);  
    benchmark("Next-Fit", next_fit);  
    return 0;  
}
```

3.2.3. Source Tugas 3

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#define NUM_FRAMES 4  
#define MAX_PAGES 100
```

```
typedef struct {  
    int page;  
    int time;  
    int dirty;  
} Frame;
```

```
typedef struct {  
    int total_access;  
    int page_fault;  
    int page_hit;  
    int disk_write;  
} PageStats;
```

```

Frame frames[NUM_FRAMES];

PageStats stats;

int clock = 0;

// ----- Utility -----

void init_system() {
    for (int i = 0; i < NUM_FRAMES; i++) {
        frames[i].page = -1;
        frames[i].time = 0;
        frames[i].dirty = 0;
    }
    stats = (PageStats){0,0,0,0};
    clock = 0;
}

// ----- FIFO -----

int fifo_replace(int page, int write) {
    int victim = 0;
    for (int i = 1; i < NUM_FRAMES; i++)
        if (frames[i].time < frames[victim].time)
            victim = i;

    if (frames[victim].dirty)
        stats.disk_write++;

    frames[victim] = (Frame){page, clock++, write};
    return victim;
}

```



```

}

// ----- LRU -----
int lru_replace(int page, int write) {
    int victim = 0;
    for (int i = 1; i < NUM_FRAMES; i++)
        if (frames[i].time < frames[victim].time)
            victim = i;

    if (frames[victim].dirty)
        stats.disk_write++;

    frames[victim] = (Frame){page, clock++, write};
    return victim;
}

// ----- Access -----
void access_page(int page, int write, int policy) {
    stats.total_access++;

    for (int i = 0; i < NUM_FRAMES; i++) {
        if (frames[i].page == page) {
            stats.page_hit++;
            frames[i].time = clock++;
            if (write) frames[i].dirty = 1;
            return;
        }
    }
}

```

```

// PAGE FAULT
stats.page_fault++;

for (int i = 0; i < NUM_FRAMES; i++) {
    if (frames[i].page == -1) {
        frames[i] = (Frame){page, clock++, write};
        return;
    }
}

if (policy == 0)
    fifo_replace(page, write);
else
    lru_replace(page, write);
}

void run_test(int policy) {
    int ref[] = {7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1};
    int n = sizeof(ref)/sizeof(ref[0]);

    init_system();

    for (int i = 0; i < n; i++)
        access_page(ref[i], 0, policy);

    printf("\nPolicy: %s\n", policy == 0 ? "FIFO" : "LRU");
    printf("Total Access  : %d\n", stats.total_access);
}

```

```

printf("Page Fault   : %d\n", stats.page_fault);
printf("Page Hit     : %d\n", stats.page_hit);
printf("Hit Ratio    : %.2f\n",
      (float)stats.page_hit / stats.total_access);
printf("Disk Writes  : %d\n", stats.disk_write);
}

```

```

int main() {
    run_test(0); // FIFO
    run_test(1); // LRU
    return 0;
}

```

3.2.4. Source Tugas 4

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

/* =====
TWO-LEVEL PAGE TABLE
===== */

#define PAGE_SIZE 4096
#define VIRTUAL_ADDR_BITS 32
#define OFFSET_BITS 12

#define OUTER_PAGE_BITS 10
#define INNER_PAGE_BITS 10

typedef struct {
    int frame_number;

```

```

    int valid;
} InnerPageTableEntry;

typedef struct {
    InnerPageTableEntry *inner_table;
    int valid;
} OuterPageTableEntry;

OuterPageTableEntry *outer_page_table;
int num_outer_entries;
int num_inner_entries;

void initialize_two_level_paging() {
    num_outer_entries = 1 << OUTER_PAGE_BITS;
    num_inner_entries = 1 << INNER_PAGE_BITS;

    outer_page_table = calloc(num_outer_entries, sizeof(OuterPageTableEntry));

    printf("=== TWO-LEVEL PAGE TABLE INITIALIZED ===\n");
}

void parse_virtual_address(unsigned int vaddr,
                           int *outer, int *inner, int *offset) {
    *offset = vaddr & ((1 << OFFSET_BITS) - 1);
    *inner = (vaddr >> OFFSET_BITS) & ((1 << INNER_PAGE_BITS) - 1);
    *outer = (vaddr >> (OFFSET_BITS + INNER_PAGE_BITS)) &
        ((1 << OUTER_PAGE_BITS) - 1);
}

void allocate_page_two_level(unsigned int vaddr, int frame) {
    int o, i, off;

```

```

parse_virtual_address(vaddr, &o, &i, &off);

if (!outer_page_table[o].valid) {
    outer_page_table[o].inner_table =
        calloc(num_inner_entries, sizeof(InnerPageTableEntry));
    outer_page_table[o].valid = 1;
}

outer_page_table[o].inner_table[i].frame_number = frame;
outer_page_table[o].inner_table[i].valid = 1;

printf("2-Level: VA 0x%X -> Frame %d\n", vaddr, frame);
}

int translate_two_level(unsigned int vaddr) {
    int o, i, off;
    parse_virtual_address(vaddr, &o, &i, &off);

    if (!outer_page_table[o].valid ||
        !outer_page_table[o].inner_table[i].valid) {
        printf("2-Level PAGE FAULT\n");
        return -1;
    }

    int frame = outer_page_table[o].inner_table[i].frame_number;
    int pa = frame * PAGE_SIZE + off;

    printf("2-Level Translation: VA 0x%X -> PA 0x%X\n", vaddr, pa);
    return pa;
}

```

```

/* =====
THREE-LEVEL PAGE TABLE
===== */

#define L1_BITS 16
#define L2_BITS 8
#define L3_BITS 8
#define OFFSET3_BITS 12

typedef struct {
    int frame;
    int valid;
    int r, w, x;
} L3_Entry;

typedef struct {
    L3_Entry *entries;
    int valid;
} L2_Entry;

typedef struct {
    L2_Entry *entries;
    int valid;
} L1_Entry;

L1_Entry *l1_table;

void init_3level() {
    l1_table = calloc(1 << L1_BITS, sizeof(L1_Entry));
    printf("\n=== THREE-LEVEL PAGE TABLE INITIALIZED ===\n");
}

```

```

void allocate_3level(unsigned long vaddr, int frame) {
    int l1 = (vaddr >> (L2_BITS + L3_BITS + OFFSET3_BITS)) & ((1 << L1_BITS) -
1);

    int l2 = (vaddr >> (L3_BITS + OFFSET3_BITS)) & ((1 << L2_BITS) - 1);
    int l3 = (vaddr >> OFFSET3_BITS) & ((1 << L3_BITS) - 1);

    if (!l1_table[l1].valid) {
        l1_table[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        l1_table[l1].valid = 1;
    }

    if (!l1_table[l1].entries[l2].valid) {
        l1_table[l1].entries[l2].entries =
            calloc(1 << L3_BITS, sizeof(L3_Entry));
        l1_table[l1].entries[l2].valid = 1;
    }

    L3_Entry *e = &l1_table[l1].entries[l2].entries[l3];
    e->frame = frame;
    e->valid = 1;
    e->r = e->w = e->x = 1;

    printf("3-Level: VA 0x%x -> Frame %d\n", vaddr, frame);
}

int translate_3level(unsigned long vaddr) {
    int offset = vaddr & ((1 << OFFSET3_BITS) - 1);
    int l1 = (vaddr >> (L2_BITS + L3_BITS + OFFSET3_BITS)) & ((1 << L1_BITS) -
1);

    int l2 = (vaddr >> (L3_BITS + OFFSET3_BITS)) & ((1 << L2_BITS) - 1);

```

```

int l3 = (vaddr >> OFFSET3_BITS) & ((1 << L3_BITS) - 1);

if (!l1_table[l1].valid ||
    !l1_table[l1].entries[l2].valid ||
    !l1_table[l1].entries[l2].entries[l3].valid) {
    printf("3-Level PAGE FAULT\n");
    return -1;
}

int frame = l1_table[l1].entries[l2].entries[l3].frame;
int pa = (frame << OFFSET3_BITS) | offset;

printf("3-Level Translation: VA 0x%lx -> PA 0x%x\n", vaddr, pa);
return pa;
}

/* =====
   INVERTED PAGE TABLE
   ===== */

#define NUM_FRAMES 64

typedef struct {
    int pid;
    int page;
    int valid;
} InvertedEntry;

InvertedEntry inverted_table[NUM_FRAMES];

int search_inverted(int pid, int page) {

```



```

    for (int i = 0; i < NUM_FRAMES; i++)
        if (inverted_table[i].valid &&
            inverted_table[i].pid == pid &&
            inverted_table[i].page == page)
            return i;
    return -1;
}

void insert_inverted(int pid, int page, int frame) {
    inverted_table[frame] = (InvertedEntry){pid, page, 1};
    printf("Inverted: PID %d Page %d -> Frame %d\n", pid, page, frame);
}

/* =====
DEMO
===== */

int main() {
    initialize_two_level_paging();
    init_3level();

    printf("\n--- TWO LEVEL DEMO ---\n");
    allocate_page_two_level(0x00001000, 5);
    translate_two_level(0x00001234);

    printf("\n--- THREE LEVEL DEMO ---\n");
    allocate_3level(0x123456789ABC, 12);
    translate_3level(0x123456789ABC);

    printf("\n--- INVERTED PAGE TABLE DEMO ---\n");
    insert_inverted(1, 10, 3);

```

```

printf("Search PID 1 Page 10 -> Frame %d\n",
      search_inverted(1, 10));

return 0;
}

```

3.2.5. Source Tugas 5

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_PROCESS 10
#define PHYSICAL_MEMORY 1024 // MB

typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int swap_count;
    time_t last_access;
} Process;

Process processes[MAX_PROCESS];
int used_memory = 0;

// ----- Initialization -----
void init_processes() {
    srand(time(NULL));

```

```

used_memory = 0;

for (int i = 0; i < MAX_PROCESS; i++) {
    processes[i].pid = i + 1;
    processes[i].size = 50 + rand() % 151; // 50–200 MB
    processes[i].priority = 1 + rand() % 5;
    processes[i].in_memory = 0;
    processes[i].swap_count = 0;
    processes[i].last_access = time(NULL);
}
}

// ----- Swap Operations -----
void swap_out(int idx) {
    processes[idx].in_memory = 0;
    used_memory -= processes[idx].size;
    processes[idx].swap_count++;
}

void swap_in(int idx) {
    processes[idx].in_memory = 1;
    used_memory += processes[idx].size;
    processes[idx].last_access = time(NULL);
}

// ----- Victim Selection -----
int select_fifo() {
    int victim = -1;

```

```

time_t oldest = time(NULL);

for (int i = 0; i < MAX_PROCESS; i++) {
    if (processes[i].in_memory &&
        processes[i].last_access <= oldest) {
        oldest = processes[i].last_access;
        victim = i;
    }
}
return victim;
}

```

```

int select_lru() {
    int victim = -1;
    time_t least = time(NULL);

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory &&
            processes[i].last_access <= least) {
            least = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

```

```

int select_priority() {
    int victim = -1;

```

```

int lowest = 999;

for (int i = 0; i < MAX_PROCESS; i++) {
    if (processes[i].in_memory &&
        processes[i].priority < lowest) {
        lowest = processes[i].priority;
        victim = i;
    }
}
return victim;
}

// ----- Simulation -----
void simulate(int strategy) {
    init_processes();
    int swap_in_count = 0, swap_out_count = 0;

    for (int t = 0; t < 100; t++) {
        int p = rand() % MAX_PROCESS;

        if (!processes[p].in_memory) {
            while (used_memory + processes[p].size > PHYSICAL_MEMORY) {
                int victim;
                if (strategy == 0) victim = select_fifo();
                else if (strategy == 1) victim = select_lru();
                else victim = select_priority();

                swap_out(victim);
            }
        }
    }
}

```

```

        swap_out_count++;
    }
    swap_in(p);
    swap_in_count++;
}
processes[p].last_access = time(NULL);
}

printf("\nStrategy: %s\n",
       strategy == 0 ? "FIFO" :
       strategy == 1 ? "LRU" : "PRIORITY");

printf("Swap In Count : %d\n", swap_in_count);
printf("Swap Out Count: %d\n", swap_out_count);
}

// ----- Main -----
int main() {
    simulate(0); // FIFO
    simulate(1); // LRU
    simulate(2); // Priority
    return 0;
}

```

3.3. Screenshot Output

3.3.1. Tugas 1

```
rafli_musthofa@RAFLI:~/tugas 7$ gcc tugas1_addreas.c -o tugas1_addreas
rafli_musthofa@RAFLI:~/tugas 7$ ./tugas1_addreas
=== ENHANCED MEMORY LAYOUT ===

PID: 1351

Text Segment:
  main() address : 0x59b1baa0b2f2
  func1() address: 0x59b1baa0b269
  func2() address: 0x59b1baa0b274
  func3() address: 0x59b1baa0b27f

Data Segment:
  global_var address      : 0x59b1baa0e010
  static_array[0] address: 0x59b1baa0e020
  string literal address  : 0x59b1baa0c0c9

Heap Segment:
  dynamic_array[0] address: 0x59b1d7eeb6b0
  malloc block 1 address  : 0x59b1d7eeb6d0
  malloc block 2 address  : 0x59b1d7eeb740

Stack Segment:
  stack_var address: 0x7ffc75a0c0fc

Stack (Recursive Calls):
  Level 3 local_var address: 0x7ffc75a0c0d4
  Level 2 local_var address: 0x7ffc75a0c0a4
  Level 1 local_var address: 0x7ffc75a0c074
  Level 0 local_var address: 0x7ffc75a0c044

Tekan Enter untuk melihat /proc/[pid]/maps...
```

```
Tekan Enter untuk melihat /proc/[pid]/maps...
59b1baa0a000-59b1baa0b000 r--p 00000000 08:30 37978      /home/rafli_musthofa/tugas 7/tugas1_addreas
59b1baa0b000-59b1baa0c000 r-xp 00001000 08:30 37978      /home/rafli_musthofa/tugas 7/tugas1_addreas
59b1baa0c000-59b1baa0d000 r--p 00002000 08:30 37978      /home/rafli_musthofa/tugas 7/tugas1_addreas
59b1baa0d000-59b1baa0e000 r--p 00002000 08:30 37978      /home/rafli_musthofa/tugas 7/tugas1_addreas
59b1baa0e000-59b1baa0f000 rw-p 00003000 08:30 37978      /home/rafli_musthofa/tugas 7/tugas1_addreas
59b1d7eeb000-59b1d7f0c000 rw-p 00000000 00:00 0          [heap]
7584acab9000-7584acadb000 r--p 00000000 08:30 41792      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7584acadb000-7584acc53000 r-xp 00022000 08:30 41792      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7584acc53000-7584acca1000 r--p 0019a000 08:30 41792      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7584acca1000-7584acca5000 r--p 001e7000 08:30 41792      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7584acca5000-7584acca7000 rw-p 001eb000 08:30 41792      /usr/lib/x86_64-linux-gnu/libc-2.31.so
7584acca7000-7584accad000 rw-p 00000000 00:00 0
7584accb6000-7584accb7000 r--p 00000000 08:30 40200      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7584accb7000-7584accda000 r-xp 00001000 08:30 40200      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7584accda000-7584acce2000 r--p 00024000 08:30 40200      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7584acce3000-7584acce4000 r--p 0002c000 08:30 40200      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7584acce4000-7584acce5000 rw-p 0002d000 08:30 40200      /usr/lib/x86_64-linux-gnu/ld-2.31.so
7584acce5000-7584acce6000 rw-p 00000000 00:00 0
7ffc759ed000-7ffc75a0e000 rw-p 00000000 00:00 0          [stack]
7ffc75a9d000-7ffc75aa1000 r--p 00000000 00:00 0          [vvar]
7ffc75aa1000-7ffc75aa3000 r-xp 00000000 00:00 0          [vdso]
rafli_musthofa@RAFLI:~/tugas 7$
```

3.3.2. Tugas 2

```
rafli_musthofa@RAFLI:~/tugas 7$ gcc tugas2_allocator.c -o tugas2_allocator
rafli_musthofa@RAFLI:~/tugas 7$ ./tugas2_allocator

[First-Fit]
Execution Time: 0 µs
Total Free Space      : 0
Largest Free Block    : 0
Free Holes             : 0
External Fragmentation: 0.00%

[Best-Fit]
Execution Time: 1 µs
Total Free Space      : 0
Largest Free Block    : 0
Free Holes            : 0
External Fragmentation: 0.00%

[Worst-Fit]
Execution Time: 1 µs
Total Free Space      : 0
Largest Free Block    : 0
Free Holes            : 0
External Fragmentation: 0.00%

[Next-Fit]
Execution Time: 1 µs
Total Free Space      : 0
Largest Free Block    : 0
Free Holes            : 0
External Fragmentation: 0.00%
rafli_musthofa@RAFLI:~/tugas 7$
```


3.3.3. Tugas 3

```
rafli_musthofa@RAFLI:~/tugas 7$ gcc tugas3_pagging.c -o tugas3_pagging
rafli_musthofa@RAFLI:~/tugas 7$ ./tugas3_pagging

Policy: FIFO
Total Access : 20
Page Fault   : 8
Page Hit     : 12
Hit Ratio    : 0.60
Disk Writes  : 0

Policy: LRU
Total Access : 20
Page Fault   : 8
Page Hit     : 12
Hit Ratio    : 0.60
Disk Writes  : 0
rafli_musthofa@RAFLI:~/tugas 7$
```

3.3.4. Tugas 4

```
rafli_musthofa@RAFLI:~/tugas 7$ ./tugas4_multilevel
=== TWO-LEVEL PAGE TABLE INITIALIZED ===

=== THREE-LEVEL PAGE TABLE INITIALIZED ===

--- TWO LEVEL DEMO ---
2-Level: VA 0x1000 -> Frame 5
2-Level Translation: VA 0x1234 -> PA 0x5234

--- THREE LEVEL DEMO ---
3-Level: VA 0x123456789abc -> Frame 12
3-Level Translation: VA 0x123456789abc -> PA 0xcabc

--- INVERTED PAGE TABLE DEMO ---
Inverted: PID 1 Page 10 -> Frame 3
Search PID 1 Page 10 -> Frame 3
```

3.3.5. Tugas 5

```
rafli_musthofa@RAFLI:~/tugas 7$ ./tugas5_swapping

Strategy: FIFO
Swap In Count : 26
Swap Out Count: 18

Strategy: LRU
Swap In Count : 26
Swap Out Count: 18

Strategy: PRIORITY
Swap In Count : 26
Swap Out Count: 19
```

3.3.6. Tugas 6

```
rafli_musthofa@RAFLI:~$ lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                2
On-line CPU(s) list:   0,1
Thread(s) per core:    1
Core(s) per socket:    2
CPU family:            23
Model:                 24
Model name:            AMD Athlon Silver 3050U with Radeon Graphics
CPU MHz:               2295.619
NUMA node0 CPU(s):     0,1
rafli_musthofa@RAFLI:~$
```

```
rafli_musthofa@RAFLI:~$ getconf PAGE_SIZE
4096
```

```

rafli_musthofa@RAFLI:~$ cpuid | grep -i tlb
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
cache and TLB information (2):
  use hypercalls for local TLB flushes = false
  use hypercalls for remote TLB flushes = true
L1 TLB/cache information: 2M/4M pages & L1 TLB (0x80000005/eax):
L1 TLB/cache information: 4K pages & L1 TLB (0x80000005/ebx):
L2 TLB/cache information: 2M/4M pages & L2 TLB (0x80000006/eax):
L2 TLB/cache information: 4K pages & L2 TLB (0x80000006/ebx):
L1 TLB information: 1G pages (0x80000019/eax):
L2 TLB information: 1G pages (0x80000019/ebx):
rafli_musthofa@RAFLI:~$

```

```

rafli_musthofa@RAFLI:~$ lscpu | grep -i cache
L1d cache: 64 KiB
L1i cache: 128 KiB
L2 cache: 1 MiB
L3 cache: 4 MiB
rafli_musthofa@RAFLI:~$

```

```

rafli_musthofa@RAFLI:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           4.8Gi        405Mi        4.2Gi        3.0Mi        178Mi        4.2Gi
Swap:          2.0Gi           0B        2.0Gi
rafli_musthofa@RAFLI:~$

```

```

rafli_musthofa@RAFLI:~$ vmstat 1 5
procs -----memory----- ---swap--  ---io---  -system--  -----cpu-----
 r b  swpd   free   buff  cache   si   so    bi   bo    in   cs us sy id wa st
 0 0     0 4394112   1196 182160    0    0   276   54   108  189  2  6 92  0  0
 0 0     0 4394112   1196 182200    0    0    0    0   35   44  0  0 100  0  0
 0 0     0 4392672   1196 182200    0    0    0   60   62   66  0  1 99  0  0
 0 0     0 4392672   1196 182200    0    0    0    0   40   43  0  0 100  0  0
 0 0     0 4392672   1196 182200    0    0    0    0   28   37  0  1 100  0  0
rafli_musthofa@RAFLI:~$

```

```

rafli_musthofa@RAFLI:~$ cat /proc/self/maps
5802b85de000-5802b85e0000 r--p 00000000 08:30 1773 /usr/bin/cat
5802b85e0000-5802b85e5000 r-xp 00002000 08:30 1773 /usr/bin/cat
5802b85e5000-5802b85e8000 r--p 00007000 08:30 1773 /usr/bin/cat
5802b85e8000-5802b85e9000 r--p 00009000 08:30 1773 /usr/bin/cat
5802b85e9000-5802b85ea000 rw-p 0000a000 08:30 1773 /usr/bin/cat
5802f56e6000-5802f5707000 rw-p 00000000 00:00 0 [heap]
76d25e88d000-76d25ea00000 r--p 00000000 08:30 22157 /usr/lib/locale/C.UTF-8/LC_COLLATE
76d25ea00000-76d25ece6000 r--p 00000000 08:30 4129 /usr/lib/locale/locale-archive
76d25ed98000-76d25edba000 rw-p 00000000 00:00 0
76d25edba000-76d25edec000 r--p 00000000 08:30 22158 /usr/lib/locale/C.UTF-8/LC_CTYPE
76d25edec000-76d25eded000 r--p 00000000 08:30 22164 /usr/lib/locale/C.UTF-8/LC_NUMERIC
76d25eded000-76d25edee000 r--p 00000000 08:30 22167 /usr/lib/locale/C.UTF-8/LC_TIME
76d25edee000-76d25edef000 r--p 00000000 08:30 22162 /usr/lib/locale/C.UTF-8/LC_MONETARY
76d25edef000-76d25edf0000 r--p 00000000 08:30 22161 /usr/lib/locale/C.UTF-8/LC_MESSAGES/SYS_LC_MESSAGES
76d25edf0000-76d25edf1000 r--p 00000000 08:30 22165 /usr/lib/locale/C.UTF-8/LC_PAPER
76d25edf1000-76d25edf2000 r--p 00000000 08:30 22163 /usr/lib/locale/C.UTF-8/LC_NAME
76d25edf2000-76d25edf3000 r--p 00000000 08:30 22156 /usr/lib/locale/C.UTF-8/LC_ADDRESS
76d25edf3000-76d25ee15000 r--p 00000000 08:30 41792 /usr/lib/x86_64-linux-gnu/libc-2.31.so
76d25ee15000-76d25ef8d000 r-xp 00022000 08:30 41792 /usr/lib/x86_64-linux-gnu/libc-2.31.so
76d25ef8d000-76d25efdb000 r--p 0019a000 08:30 41792 /usr/lib/x86_64-linux-gnu/libc-2.31.so
76d25efdb000-76d25efd0000 r--p 001e7000 08:30 41792 /usr/lib/x86_64-linux-gnu/libc-2.31.so
76d25efd0000-76d25efe1000 rw-p 001eb000 08:30 41792 /usr/lib/x86_64-linux-gnu/libc-2.31.so
76d25efe1000-76d25efe7000 rw-p 00000000 00:00 0
76d25efe7000-76d25efe8000 r--p 00000000 08:30 22166 /usr/lib/locale/C.UTF-8/LC_TELEPHONE
76d25efe8000-76d25efe9000 r--p 00000000 08:30 22160 /usr/lib/locale/C.UTF-8/LC_MEASUREMENT
76d25efe9000-76d25eff0000 r--s 00000000 08:30 42059 /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
76d25eff0000-76d25eff1000 r--p 00000000 08:30 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
76d25eff1000-76d25f014000 r-xp 00001000 08:30 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
76d25f014000-76d25f01c000 r--p 00024000 08:30 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
76d25f01c000-76d25f01d000 r--p 00000000 08:30 22159 /usr/lib/locale/C.UTF-8/LC_IDENTIFICATION
76d25f01d000-76d25f01e000 r--p 0002c000 08:30 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
76d25f01e000-76d25f01f000 rw-p 0002d000 08:30 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
76d25f01f000-76d25f020000 rw-p 00000000 00:00 0
7ffe907d2000-7ffe907f3000 rw-p 00000000 00:00 0 [stack]
7ffe907fa000-7ffe907fe000 r--p 00000000 00:00 0 [vvar]
7ffe907fe000-7ffe90800000 r-xp 00000000 00:00 0 [vdso]

```

```

rafli_musthofa@RAFLI:~$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sdc  partition 2G    0B   -2
rafli_musthofa@RAFLI:~$

```

3.4. Penjelasan Implementasi

3.4.1. Tugas 1

Tugas 1 di implementasikan menggunakan program bahasa C yang bertujuan untuk mengamati pembagian memori proses linux, meliputi:

- Text Segment : alamat fungsi
- Data Segment : global variabel terinisialisasi
- BSS segment : global variabel tidak terinisialisasi
- Heap Segment : memori hasil malloc()
- Stack Segment : variabel lokal fungsi

Program juga:

- Menampilkan PID proses
- Menampilkan memory map proses melalui /proc/[pid]/maps

Hasil Implementasi

- Setiap segmen berada pada alamat memori yang berbeda
- Heap bertumbuh ke atas, stack ke bawah
- Linux memisahkan segment secara jelas demi proteksi memori

3.4.2. Tugas 2

Tugas 2 mengimplementasikan simulator alokasi memori kontigu menggunakan empat algoritma:

1. First-Fit
2. Best-Fit
3. Worst-Fit
4. Next-Fit

Fitur utama:

- Alokasi dan dealokasi proses
- Pemecahan block memori
- Penggabungan block (merge)
- Menampilkan status memori (fragmentasi)

Hasil Implementasi

- First-Fit cepat tetapi mudah fragmentasi
- Best-Fit mengurangi sisa kecil tapi mahal komputasi
- Worst-Fit menghasilkan fragmentasi besar
- Next-Fit lebih adil dibanding First-Fit

3.4.3. Tugas 3

Tugas 3 Implementasi sebagai paging simulator dengan:

- Page table
- Physical memory (frame)
- Referince string
- Page fault counter

Algoritma page replacement:

- FIFO
- LRU

Program menampilkan:

- Status frame setiap akses
- Total page fault
- Perbandingan performa algoritma

Hasil Implementasi

- FIFO sederhana tetapi rentan Belady's Anomaly
- LRU lebih optimal namun kompleks
- Paging menghilangkan fragmentasi eksternal

3.4.3. Tugas 4 – Multi Level Page Table

Tugas 4 mengimplementasikan Two Level Page Table dengan:

- Outer page table
- Inner page table
- Translasi alamat virtual – fisik
- Perhitungan memory overhead

Program mendukung:

- Alokasi page

- Address translation
- Analisis overhead single-level vs multilevel

Hasil Implementasi

- Two lwwl paging jauh lebih hemat memori
- Inner table hanya dibuat saat dibutuhkan
- Digunakan pada sistem modern (x86-64, ARM)

3.4.5. Tugas 5 – Swapping Simulation

Tugas 5 mensimulasikan swapping antara RAM dan disk:

- Ready queue & swap space
- Load proccess ke memory
- Swap-out saat memory penuh
- Swap-in saat proses dijalankan

Simulasi menampilkan

- Proses ram
- Proses di swap
- Waktu eksekusi

Hasil Implementasi

- Swapping memungkinkan multitasking pada RAM terbstsd
- Terlalu sering sapping menyebabkan thrashing
- Swap disk jauh lebih lambat dibanding RAM

3.4.6. Tugas 6

Tugas 6 dilakukan dengan analisis sistem nyata Ubuntu, menggunakan command linux:

- Lscpu
- Getconf PAGE_SIZE
- Cpuuid
- Free – h
- Vsmtat
- /proc/self/maps
- Swapon – show

BAB IV ANALISIS DAN PEMBAHASAN

4.1. Analisis Tugas 1

Analisis:

Pada Tugas 1 dilakukan pengamatan terhadap tata letak memori suatu proses pada sistem operasi Linux. Program C menampilkan alamat dari berbagai segmen memori, yaitu text, data, BSS, heap, dan stack. Hasil pengujian menunjukkan bahwa setiap segmen berada pada alamat yang berbeda dan terisolasi satu sama lain.

Text segment memiliki alamat tetap selama program berjalan karena berisi instruksi program. Data segment dan BSS berada pada area memori global yang dialokasikan saat proses dimuat. Heap dialokasikan secara dinamis menggunakan malloc() dan bertumbuh ke arah alamat yang lebih besar, sedangkan stack menyimpan variabel lokal dan bertumbuh ke arah alamat yang lebih kecil.

Pembahasan:

Pemisahan segmen memori ini bertujuan untuk meningkatkan keamanan dan stabilitas sistem. Kesalahan pada satu segmen, seperti stack overflow, tidak secara langsung merusak segmen lain. Selain itu, pemetaan memori yang terlihat pada /proc/[pid]/maps menunjukkan bagaimana Linux menerapkan proteksi memori berbasis hardware.

4.2. Analisis Tugas 2

Analisis:

Tugas 2 mengimplementasikan beberapa algoritma alokasi memori kontigu, yaitu First-Fit, Best-Fit, Worst-Fit, dan Next-Fit. Setiap algoritma memiliki strategi berbeda dalam memilih blok memori kosong untuk proses baru.

Hasil simulasi menunjukkan bahwa First-Fit memiliki waktu pencarian yang cepat, tetapi cenderung menghasilkan fragmentasi eksternal. Best-Fit mengurangi sisa memori kecil, namun memerlukan waktu pencarian lebih lama. Worst-Fit meninggalkan blok besar tetapi sering menyebabkan pemborosan. Next-Fit mencoba mendistribusikan alokasi lebih merata.

Pembahasan:

Fragmentasi eksternal menjadi masalah utama pada alokasi kontigu. Hal ini menunjukkan keterbatasan pendekatan contiguous memory allocation dan menjadi alasan utama berkembangnya teknik paging dan segmentation pada sistem operasi modern.

4.3. Analisis Tugas 3

Analisis:

Pada Tugas 3 dilakukan simulasi sistem paging beserta algoritma page replacement FIFO dan LRU. Sistem akan menghasilkan page fault ketika page yang diakses tidak berada di memori fisik.

FIFO mengganti page yang paling lama berada di memori tanpa mempertimbangkan pola akses. LRU mengganti page yang paling jarang digunakan berdasarkan histori akses.

Hasil simulasi menunjukkan bahwa FIFO dapat mengalami fenomena Belady's Anomaly, di mana penambahan jumlah frame justru meningkatkan jumlah page fault. Sebaliknya, LRU menghasilkan performa lebih stabil.

Pembahasan:

Paging menghilangkan fragmentasi eksternal dan memungkinkan proses memiliki ruang alamat yang lebih besar dari memori fisik. Namun, paging membutuhkan hardware tambahan seperti TLB untuk mempercepat translasi alamat.

4.4. Analisis Tugas 4

Analisis:

Tugas 4 mengimplementasikan two-level page table untuk mengurangi overhead memori akibat page table yang besar. Inner page table hanya dialokasikan ketika diperlukan.

Perbandingan menunjukkan bahwa multi-level page table jauh lebih hemat memori dibandingkan single-level page table, terutama ketika address space sangat besar namun hanya sebagian kecil yang digunakan.

Pembahasan:

Pendekatan multi-level paging merupakan solusi praktis pada arsitektur 64-bit. Walaupun menambah langkah translasi, overhead tersebut dikompensasi oleh TLB sehingga performa tetap tinggi.

4.5. Analisis Tugas 5

Analisi:

Pada Tugas 5 dilakukan simulasi swapping antara memori utama dan disk. Ketika memori penuh, proses yang jarang digunakan dipindahkan ke swap space.

Hasil simulasi menunjukkan bahwa swapping memungkinkan sistem menjalankan lebih banyak proses daripada kapasitas RAM yang tersedia, namun dengan konsekuensi penurunan performa.

Pembahasan:

Swapping efektif pada kondisi beban ringan hingga sedang. Namun, jika terlalu sering terjadi swapping, sistem dapat mengalami thrashing, di mana sebagian besar waktu digunakan untuk memindahkan data, bukan menjalankan proses.

4.6. Analisis Tugas 6

Analisis:

Pada Tugas 6 dilakukan analisis arsitektur memori pada sistem Ubuntu berbasis x86-64. Hasil pengujian menunjukkan bahwa sistem menggunakan page size 4 KB dengan multi-level paging dan dukungan TLB untuk mempercepat address translation.

Cache hierarchy terdiri dari L1, L2, dan L3 cache yang berfungsi mengurangi latency akses memori. Sistem juga menggunakan swap sebagai ekstensi memori fisik.

Pembahasan:

Arsitektur x86-64 dirancang untuk mendukung ruang alamat yang sangat besar dan multitasking intensif. Implementasi multi-level paging dan cache hierarchy memungkinkan sistem tetap efisien meskipun kompleksitas meningkat.

BAB V KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil implementasi, analisis, dan pembahasan pada Tugas 1 sampai dengan Tugas 6, dapat disimpulkan bahwa manajemen memori merupakan komponen yang sangat penting dalam sistem operasi modern. Setiap teknik manajemen memori dirancang untuk mengatasi permasalahan tertentu yang muncul akibat keterbatasan sumber daya memori.

Pada **Tugas 1**, diperoleh pemahaman bahwa sistem operasi Linux membagi memori proses ke dalam beberapa segmen seperti text, data, BSS, heap, dan stack. Pemisahan ini bertujuan untuk meningkatkan keamanan, efisiensi, serta kemudahan pengelolaan memori proses.

Pada **Tugas 2**, simulasi alokasi memori kontigu menunjukkan bahwa algoritma First-Fit, Best-Fit, Worst-Fit, dan Next-Fit memiliki karakteristik dan kelemahan masing-masing. Fragmentasi eksternal menjadi masalah utama pada metode alokasi kontigu sehingga pendekatan ini kurang efisien untuk sistem berskala besar.

Pada **Tugas 3**, penerapan paging dan page replacement membuktikan bahwa paging mampu menghilangkan fragmentasi eksternal. Algoritma FIFO dan LRU menunjukkan perbedaan performa, di mana LRU cenderung lebih efisien karena mempertimbangkan pola akses memori, sedangkan FIFO rentan terhadap Belady's Anomaly.

Pada **Tugas 4**, penggunaan multi-level page table terbukti mampu mengurangi overhead memori dibandingkan single-level page table. Teknik ini sangat relevan pada arsitektur 64-bit dengan ruang alamat virtual yang besar dan banyak digunakan pada sistem operasi modern.

Pada **Tugas 5**, simulasi swapping menunjukkan bahwa mekanisme ini memungkinkan sistem menjalankan lebih banyak proses meskipun memori fisik terbatas. Namun, penggunaan swapping yang berlebihan dapat menurunkan performa sistem dan menyebabkan thrashing.

Pada **Tugas 6**, analisis arsitektur sistem nyata menunjukkan bahwa sistem Ubuntu berbasis x86-64 menggunakan multi-level paging, TLB, cache hierarchy, dan swap untuk

mengelola memori secara efisien. Hal ini membuktikan bahwa manajemen memori modern merupakan kombinasi antara teknik perangkat lunak dan dukungan perangkat keras.

Secara keseluruhan, praktikum ini memberikan pemahaman komprehensif mengenai konsep, implementasi, serta trade-off dalam manajemen memori pada sistem operasi.

5.2. Saran

Berdasarkan hasil praktikum yang telah dilakukan, terdapat beberapa saran yang dapat diberikan, antara lain:

1. Praktikum selanjutnya dapat dikembangkan dengan menambahkan simulasi page replacement lanjutan seperti Optimal dan Clock untuk perbandingan yang lebih luas.
2. Analisis performa dapat diperkuat dengan penggunaan benchmark nyata atau pengujian pada beban sistem yang berbeda.
3. Implementasi dapat dikembangkan menggunakan bahasa pemrograman lain atau tools visual untuk mempermudah pemahaman alur manajemen memori.
4. Disarankan untuk melakukan eksperimen langsung pada berbagai arsitektur perangkat keras, seperti ARM, guna memperluas pemahaman tentang perbedaan implementasi manajemen memori.

5.3. Kendala yang dihadapi

1. Pemahaman konsep manajemen memori

Pada tahap awal, terdapat kesulitan dalam memahami perbedaan antara berbagai teknik manajemen memori seperti alokasi kontigu, paging, multi-level page table, dan swapping. Setiap teknik memiliki mekanisme dan tujuan yang berbeda sehingga memerlukan pemahaman konseptual yang mendalam sebelum dapat diimplementasikan.

2. Kesalahan Implementasi program

Dalam proses pembuatan program simulasi, beberapa kesalahan seperti kesalahan logika, kesalahan indeks array, dan kesalahan pengelolaan memori dinamis sering terjadi. Hal ini menyebabkan program tidak berjalan sesuai harapan atau menghasilkan output yang tidak tepat.

3. Penggunaan perintah Linux

Pada Tugas 6, kendala muncul saat menjalankan perintah Linux tertentu seperti `cpuid` dan `vmstat` karena tidak semua perintah tersedia secara default atau memerlukan instalasi tambahan serta hak akses tertentu.

4. Interpretasi Output sistem

Output dari perintah seperti `lscpu`, `/proc/self/maps`, dan `vmstat` cukup kompleks dan memerlukan analisis lebih lanjut agar dapat dipahami dengan benar serta dikaitkan dengan konsep manajemen memori yang telah dipelajari.

DAFTAR PUSTAKA

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Hoboken, NJ: John Wiley & Sons.
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Boston: Pearson Education.
3. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson Education.
4. Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley Professional.
5. Intel Corporation. (2023). *Intel® 64 and IA-32 Architectures Software Developer's Manual*.
<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
6. Ubuntu Documentation. (2024). *Memory Management and Virtual Memory*.
<https://help.ubuntu.com>
7. Kerrisk, M. (2010). *The Linux Programming Interface*. No Starch Press.
8. Love, R. (2019). *Linux System Programming*. O'Reilly Media.

